YAPL-S Language Specification

YAPL Extended with Native String Operations
CS 327: Compilers

Group 6
21110202 21110060 22110099 21110210

Spring 2026

Contents

1 Introduction

YAPL-S is a variant of the YAPL language (a subset of ANSI 2011 C Standard) as taught in **CS327: Compilers** course at IITGN. It retains the core syntax and semantics of YAPL (language constructs, supported data types and supported operators), but additionally introduces native support for basic string manipulation as YAPL does not support `string` preprocessors.

This specification details the extensions made to the base YAPL language to support Pythonic **F-Strings** (Format Strings) and **String Concatenation** as native language features, eliminating the need for the `<string.h>` library for common string operations.

Note: A summary of the original YAPL language specification is provided in Appendix A.

2 Lexical Conventions

The lexical analyzer distinguishes tokens based on the standard C rules, with the following additions for handling string interpolation.

2.1 String Concatenation Operator

The following operator is added to the language:

Operator	Symbol	Description
Concatenation	`@`	Binary operator for joining two string literals.

Table 1: New Operators in YAPL-S

2.2 Interpolated String Literals

An interpolated string literal (Pythonic F-String) is distinguished from a standard string literal by the prefix `f`.

Syntax:

```
f" text { expression } text "
```

The lexical analyzer would process F-Strings using the `lex` tool to emit the following distinct token types:

- `FSTRING_START`: The sequence `f"`.
- `STRING_LITERAL_PART`: A sequence of characters inside the F-String that are *not* part of an interpolation.
- `INTERPOLATION_START`: The { character inside an F-String.
- `INTERPOLATION_END`: The } character inside an F-String.
- `FSTRING_END`: The closing " character.

Note: Standard escape sequences (e.g., `\n`, `\"`) are valid within `STRING_LITERAL_PART`.

3 Syntax

The grammar of YAPL-S extends the YAPL grammar.

3.1 Operator Precedence and Associativity

The `@` operator is inserted strictly between the **Shift** and **Relational** tiers. This ensures `char *` arithmetic (addition/subtraction) takes priority over concatenation.

Prec.	Operator Class	Operators	Associativity
1	Primary/Postfix	`() [] -> .`	L-to-R
2	Unary	`* & - !`	R-to-L
3	Multiplicative	`* / %`	L-to-R
4	Additive	`+ -`	L-to-R
5	Shift	`<< >>`	L-to-R
6	**Concatenation**	`@`	**L-to-R**
7	Relational	`< > <= >= <=>`	L-to-R
8	Equality	`== !=`	L-to-R

Table 2: Revised Operator Precedence Table

3.2 Grammar Specification

The grammar hierarchy reflects the precedence table.

```
// 1. Relational expressions derive from concatenation
relational_expression
  : concatenation_expression
  | relational_expression '<' concatenation_expression
  | relational_expression '>' concatenation_expression
  | relational_expression LE_OP concatenation_expression
  | relational_expression GE_OP concatenation_expression
  | relational_expression TH_OP concatenation_expression
  ;

// 2. Concatenation derives from Shift
// This ensures @ has lower precedence than <<, +, *, etc.
concatenation_expression
    : shift_expression
    | concatenation_expression '@' shift_expression
    ;

// 3. Shift derives from Additive (Standard C)
shift_expression
  : additive_expression
  | shift_expression LEFT_OP additive_expression
  | shift_expression RIGHT_OP additive_expression
  ;

// 4. Primary Expression updated for F-Strings
primary_expression
  : IDENTIFIER
  | constant
  | string
  | '(' expression ')'
  | generic_selection
    | f_string_expression   /* Extension */
  ;

```

```
// 5. F-String Structure
f_string_expression
    : FSTRING_START f_string_body FSTRING_END
    ;

f_string_body
    : /* empty */
    | f_string_body STRING_LITERAL_PART
    | f_string_body interpolation_block
    ;

interpolation_block
    : INTERPOLATION_START expression INTERPOLATION_END
    ;
```

4 Semantics

4.1 F-String Evaluation

An F-String expression evaluates to a value of type `char *`.

1. **Evaluation Order:** The expressions inside the interpolation blocks $\{\ldots\}$ are evaluated from left to right at runtime.
2. **Type Conversion:** The result of each interpolated expression is implicitly converted to its string representation.
   - `int`: Converted to decimal string (e.g., `10` $\rightarrow$ `"10"`).
   - `float/double`: Converted to standard floating-point representation.
   - `char *`: Copied as-is.
   - `char`: Converted to a single-character string.
3. **Construction:** The literal text parts and the converted expression strings are concatenated in order.
4. **Memory:** The result is stored in a newly allocated memory block (heap). The runtime environment manages this allocation.

4.2 The Concatenation Operator (`@`)

The `@` operator performs string concatenation.

- **Operands:** Both operands must evaluate to type `char *`.
- **Result:** A new `char *` containing the contents of the left operand followed immediately by the right operand.
- **Error Handling**: The compiler shall issue a Type Error if an operand is numeric (e.g., `str @ 5` is illegal).
- **Behavior**: Resultant string is automatically null-terminated.
  1. Compute length of left string (L_1) and right string (L_2).
  2. Allocate $L_1 + L_2 + 1$ bytes on the heap.
  3. Copy Left String $\rightarrow$ Copy Right String $\rightarrow$ Null Terminate.

- **Rationale for Precedence**: Given `str1 @ str2 + 1`:
  - Additive (+) runs first: `str2 + 1` advances the pointer by 1 char.
  - Concatenation (@) runs second: Joins `str1` with the substring.
  - This prevents the inefficiency of allocating a new string only to immediately skip its first character.

5 Examples and Use Cases

5.1 Initialization and Assignment

Native strings can be created dynamically without reliance on `sprintf` or `strcat`.

```
int main() {
    int id = 42;
    double score = 98.5;
    char *user = "Alice";

    // --- Standard C11 Approach (Tedious) ---
    // Requires <stdio.h>, <stdlib.h>, <string.h>
    // char buffer[100];
    // sprintf(buffer, "User: %s (ID: %d)", user, id);

    // --- YAPL-S Approach (Native) ---
    // F-Strings handle implicit memory allocation and formatting
    char *status = f"User: {user} (ID: {id})";

    // String Concatenation using the @ operator
    char *full_msg = status @ " - Status: Active";

    // full_msg is now: "User: Alice (ID: 42) - Status: Active"

    return 0;
}
```

5.2 Usage in Function Calls

F-Strings evaluate to `char *`, allowing them to be passed directly to any function expecting a C string.

```
void log_event(char *msg) {
    // Implementation for logging...
}

int main() {
    int x = 10, y = 20;

    // Direct usage in printf (replaces %s format specifiers)
    printf(f"Coordinates: x={x}, y={y}\n");

    // Usage in custom functions with embedded arithmetic
    // The expression {x + y} is evaluated before string construction
    log_event(f"System Check: {x + y} units detected.");

    return 0;
}
```

5.3 Interaction with Pointer Arithmetic

Because `@` has lower precedence than additive operators (`+`, `-`), standard C pointer arithmetic behaves predictably when mixed with concatenation.

```
int main() {
    char *h = "Hello";
    char *w = "World";

    // Scenario A: Pointer Arithmetic on the Operand (Standard Precedence)
    // 1. ("World" + 1) advances pointer to "orld"
    // 2. "Hello" @ "orld" concatenates
    char *subset = h @ w + 1;
    // Result: "Helloorld"

    // Scenario B: Pointer Arithmetic on the Result (Grouped)
    // 1. ("Hello" @ "World") creates "HelloWorld"
    // 2. (+ 1) advances pointer on the new string
    char *offset = (h @ w) + 1;
    // Result: "elloWorld"

    return 0;
}
```

5.4 Nested Expressions and Escaping

Based on the implementation, F-strings can be nested arbitrarily deep, enabling complex formatting logic.

```
int main() {
    int val = 100;

    // Nested F-String:
    // The inner expression { f"[{val}]" } evaluates to "[100]" first.
    char *debug = f"Debug Info: { f"Value -> [{val}]" }";

    return 0;
}
```

5.5 Compiler Error Handling

The compiler enforces strict type safety for the `@` operator to prevent accidental pointer miscalculations.

```
int main() {
    char *base = "Error Code: ";
    int code = 404;

    // INVALID: Cannot concatenate string (char *) with integer (int)
    // char *err = base @ code;  <-- Compilation Error

    // CORRECT: Use F-String to convert the integer first
    char *msg = base @ f"{code}";
    // Result: "Error Code: 404"

    return 0;
}
```

A Appendix: Base YAPL Language Specification

This appendix details the specification for the base YAPL language (a subset of C11) as available here: YAPL Language Specification (PDF).

A.1 Language Constructs

The language conforms to the ANSI C-2011 standard with the following specific inclusions:

- global variables, both normal as well as `extern`.
- function declaration, function definition, and function call (including recursion).
- `for` loop, `while` loop, `do-while`.
- `if`, `if-else`, `if-else-if` (nested/ladder) (if condition can be any expression including literals/constants).
- `switch/case` with case labeled with `:`; must support fall-through; must support the optional `default` clause.
- variable declaration, definition, initialization locally/globally (multiple variables comma-separated).
- strings (of the forms `"..."` (single), `"..." "..."` (multiple)), character, integer and floating point literals.
- single-line (`//`) and multi-line (`/*...*/`) comments (multi-line comments should be properly closed lexically).
- `break`, `continue`, and `return` statements.
- a statement must end in a semi-colon `;` like the usual standard of C.

A.2 Supported Data Types

- **Primary:** `int`, `long`, `short`, `float`, `double`, `void`, `char` and their pointers.
- **User-defined:** structures (`struct`) (pointer dereferences must support arbitrary level of indirection).
- **Derived:** functions, arrays, and pointers (all supported data types; no need to support function pointers).

A.3 Supported Operators

Operator precedence and associativity follow standard C rules.

- **Relational operators:** `<`, `>`, `==`, `<=`, `>=`, `!=`, `<=>`.
- **Unary operators:** `+`, `-`, `!`, `*`, `&`, `~`.
- **Binary operators:** `+`, `-`, `*`, `&`, `|`, `/`, `%`, `^`.
- **Logical operators:** `&&`, `||`.
- **Assignment operator:** `=`.
- **Suffix/postfix increment and decrement:** `++`, `--`.

- **Structure field access operators:** `->`, `.`.
- **Pointers:** `*`, `&` (pointer dereferences must support arbitrary/multiple level of indirection, i.e., `int *****p;`).
- **Function call:** `()` (any valid expression inside including blank, constants, and literals).
- **Array subscripting:** `[]` (any valid expression inside including constants and literals).
- `sizeof()`**:** operands: `<typename>` or unary expression.
- **Typecast:** `(<typename>)`.

A.4 Unsupported Constructs

The following C11 features are **explicitly excluded**:

- **Preprocessors:** none allowed (no `#include`s, `#define`s needed anywhere).
- **Operators:** `...` (ellipsis), `?:` (ternary operator) (function vararg list specification not required).
- **Operators:** `>>=`, `<<=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `^=`, `|=`, `<<`, `>>`.
- **Constructs/keywords:** `auto`, `const`, `goto`, `inline`, `register`, `restrict`, `signed`, `static`, `typedef`, `unsigned`, `enum`, `union`, `volatile`, `_Alignas`, `_Alignof`, `_Atomic`, `_Bool`, `_Complex`, `_Generic`, `_Imaginary`, `_Noreturn`, `_Static_assert`, `_Thread_local`, `__func__`.